AGENT EVOLUTION · NOTEBOOK 7 OF 8

Version 7 — Middleware: Clean Tool Output

A guide to `version7_budget_and_tooloutput_middleware.ipynb`: a second tool wrapper that rewrites `run_query` output before the model reads it, how two wrappers nest, and why their order matters.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Stack: LangChain `create_agent` · `AgentMiddleware` · `wrap_tool_call`

Tools: same 6 · Prompt: unchanged · Middleware: `SQLBudgetMiddleware` (outer) + `ToolOutputMiddleware` (inner)

1. What changes from Version 6
2. Two wrappers, nested
3. Order: first in the list is outermost
4. Inside `ToolOutputMiddleware`
5. The cleaning rules
6. Why cut long output
7. ToolMessage or Command
8. Testing the rules offline
9. One run, step by step
10. What the cleaner does not do
11. Two middleware, two jobs
12. The limitation, and what Version 8 changes
13. Key takeaways

1 What changes from Version 6

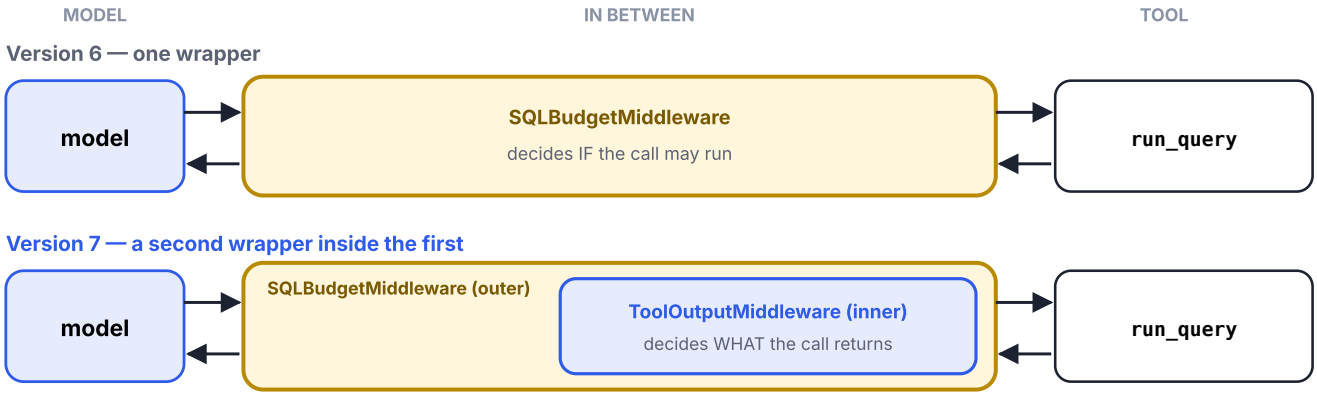


Figure 1. Same model, tools and prompt. A second wrapper is added inside the budget.

	Version 6	Version 7
Tools / prompt	6 / Version 5 prompt	**unchanged** / **unchanged**
Middleware	`SQLBudgetMiddleware`	`SQLBudgetMiddleware` + `ToolOutputMiddleware`
What reaches the model from `run_query`	raw tool text	cleaned text: clear errors, empty flagged, long output cut with a marker

Wrapper moves used	pass through · block · add to result	+ **rewrite** the result

2 Two wrappers, nested

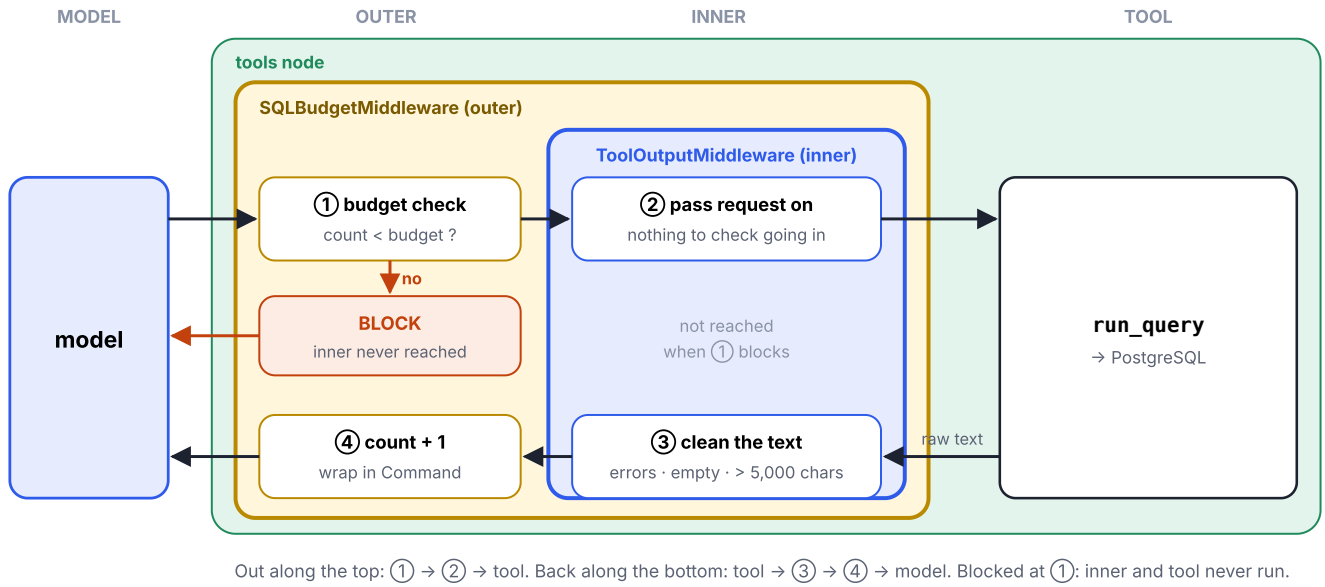


Figure 2. The key picture. The inner wrapper sits closest to the tool, so it sees the raw output first.

Each wrapper's `handler` is the next layer in. For the outer budget, `handler` runs the inner cleaner. For the inner cleaner, `handler` runs the real tool.

1. ① **Budget check (outer, going in).** Over budget → BLOCK, and nothing further in runs.
2. ② **Inner, going in.** The cleaner has nothing to check yet; it calls its `handler`, which runs `run_query`.
3. ③ **Clean (inner, coming out).** The raw text reaches the cleaner first. It rewrites errors, flags empty output, and cuts long output.
4. ④ **Count (outer, coming out).** The budget wraps the cleaned `ToolMessage` in a `Command` that adds 1 to `sql_call_count`.

> Checked on the real library with a budget of 1. The inner cleaner saw every lookup and validation call (and passed them through), and only the first `run_query`. The second `run_query` was blocked by the outer budget and never reached the inner cleaner.

3 Order: first in the list is outermost

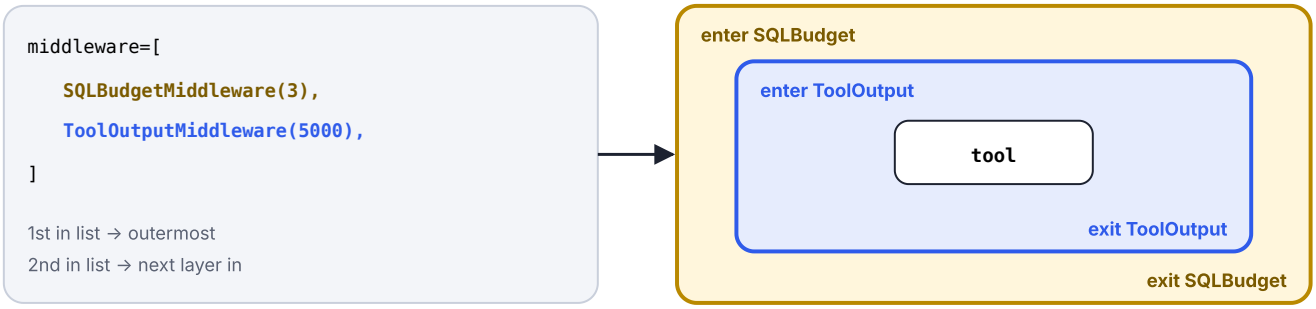


Figure 3. The list order becomes the nesting order. Entry runs outside-in; exit runs inside-out.

LangChain nests middleware in list order. A test with two logging wrappers on the real library printed:

```
enter FIRST → enter SECOND → tool → exit SECOND → exit FIRST
```

So with the notebook's list, the budget checks first on the way in, and the cleaner works first on the way out. Swapping the order changes behaviour:

	Budget outer (notebook)	Cleaner outer (swapped)
Blocked call	stopped before the cleaner; block message goes straight to the model	the block message itself passes through the cleaner
What the cleaner receives	the tool's raw `ToolMessage`	the budget's `Command`, which it must open to find the message
Reading the code	"may it run?" then "what does it return?"	harder to follow

> Rule of thumb: put **gatekeepers** (allow / block) outermost, so a blocked call costs nothing. Put **transformers** (rewrite the result) closest to the tool, so they see the raw output first.

4 Inside `ToolOutputMiddleware`

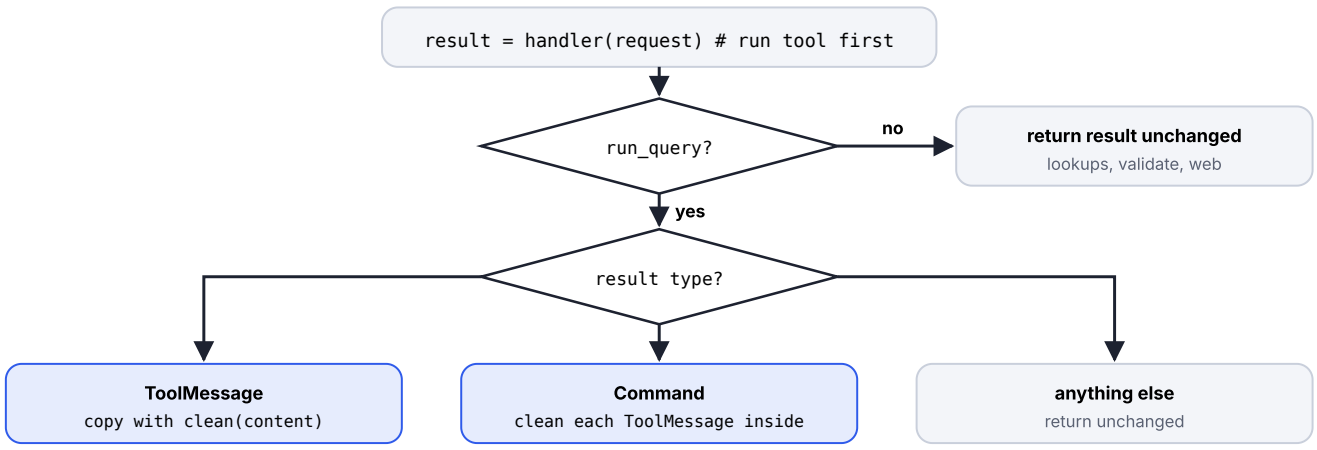


Figure 4. `_process(request, result)`: the logic shared by the sync and async hooks.

```
def wrap_tool_call(self, request, handler):
    return self._process(request, handler(request))
```

```
async def awrap_tool_call(self, request, handler):
    return self._process(request, await handler(request))
```

This wrapper uses only one move: **change the result**. It always calls `handler` first, then looks at what came back. It never blocks anything. Both hooks are one line each and share `_process`, so sync and async can never behave differently.

5 The cleaning rules

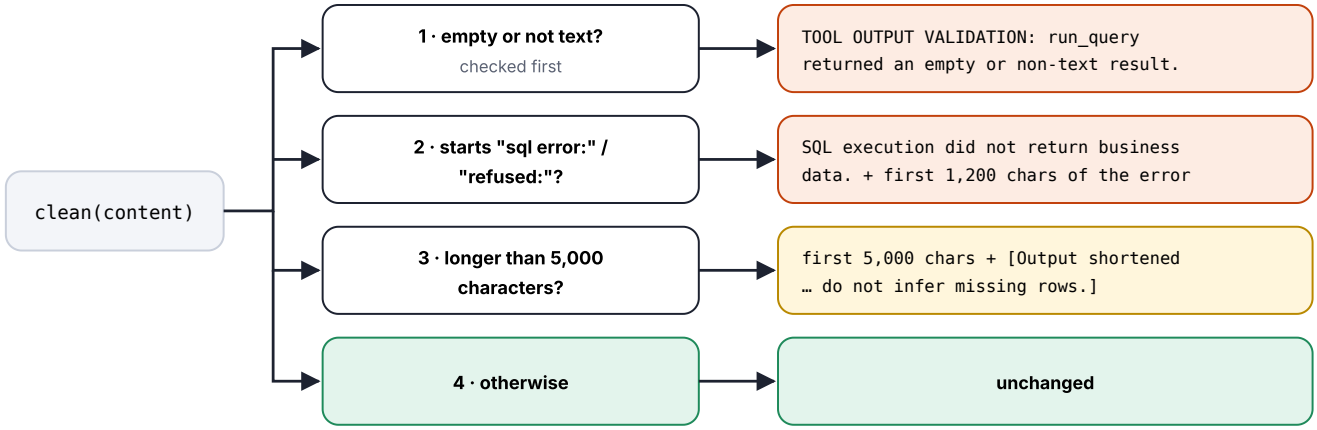


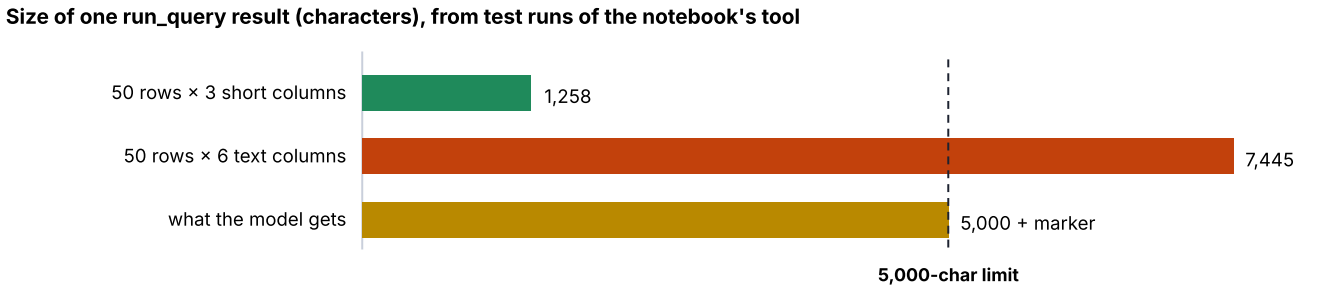
Figure 5. The rules are checked in order; the first match wins.

Real outputs of `clean()` from the notebook's code:

Raw `run_query` text	What the model receives
`""`	`TOOL OUTPUT VALIDATION: run_query returned an empty or non-text result.`
`SQL error: column "revnue" does not exist`	`SQL execution did not return business data. SQL error: column "revnue" does not exist`
`REFUSED: Only read-only SELECT or WITH queries are allowed.`	`SQL execution did not return business data. REFUSED: Only read-only …`
a 6,000-character result	first 5,000 characters + `[Output shortened to 5000 characters. The displayed result may be incomplete; do not infer missing rows.]`
`status | count / cancelled | 7 / (1 rows.)`	unchanged

Each rule adds a **plain statement of what happened** in front of the raw text. “SQL execution did not return business data” tells the model, before it reads anything else, that there are no numbers here to report.

6 Why cut long output



Every later pass resends the whole message list. A 7,445-character result in pass 3 is sent again in pass 4, 5, ...
The 50-row cap limits rows; the character limit catches results that are wide rather than long.

Figure 6. The 50-row cap from Version 2 is not enough on its own. Wide rows can still produce large results.

`run_query` already returns at most 50 rows. That keeps narrow results small: 50 rows of three short columns came to 1,258 characters. But rows can be wide. 50 rows of six text columns came to 7,445 characters, and a `SELECT *` on a wide table can be much larger.

- **Cost and speed:** the result stays in the messages and is resent on every later model call.
- **Focus:** a long block of rows pushes the instructions and earlier results further away in the context.
- **Honesty:** the marker says the table is incomplete and tells the model **not to infer missing rows**. Without it, the model might treat the first part as the whole result.

> 5,000 is a character count, not a token count, and the cut is made at exactly 5,000 characters. It can stop in the middle of a row. The marker is what makes that safe: the model is told the last line may be cut.

7 ToolMessage or Command

Version 7: run_query returns text

```
ToolMessage(
   content="status | count …"
)
```

→ clean(content), copy the message

Version 8: run_query returns a Command

```
Command(update={
   "messages": [ToolMessage(…)],
   "captured_result": {…},
   "executed_sql": "SELECT …"})
```

→ clean each ToolMessage inside messages, keep every other key as it is

Figure 7. The cleaner handles both shapes, so Version 8 can reuse it unchanged.

A tool, or an inner wrapper, can return two shapes: a plain `ToolMessage`, or a `Command` that carries a `ToolMessage` plus state updates. In Version 7, the inner cleaner always receives a plain `ToolMessage` (the budget wraps its `Command` only after the cleaner returns). But in Version 8, `run_query` itself returns a `Command` so it can save rows to state.

Handling both now means Version 8 reuses this class without a single change. For a `Command`, the cleaner rewrites only the `ToolMessage` text inside `messages` and passes every other key through untouched, so the rows saved for the application stay complete even when the text the model sees is cut.

8 Testing the rules offline

```
cleaner = ToolOutputMiddleware(max_output_chars=200)

print(cleaner.clean(""))
print(cleaner.clean('SQL error: column "revnue" does not exist'))
long_table = "product_id | revenue\n" + "\n".join(f"{i} | {i * 10.5}" for i in range(100))
print(cleaner.clean(long_table))
```

The text rules live in a plain method, `clean()`, separate from the hooks. That makes them testable with no agent, no model and no database. The notebook sets a small limit (200) so the cut is easy to see on screen.

Design choice	Benefit
`clean()` is a public method	test the rules directly, in one line each
`_process()` holds the type handling	both hooks share it; one place to fix
limit is a constructor argument	tests use 200; the agent uses 5,000

The same split was used in Version 6: decision helpers separate from the hooks. Keeping rules out of the hook functions is what makes middleware easy to test and change.

9 One run, step by step

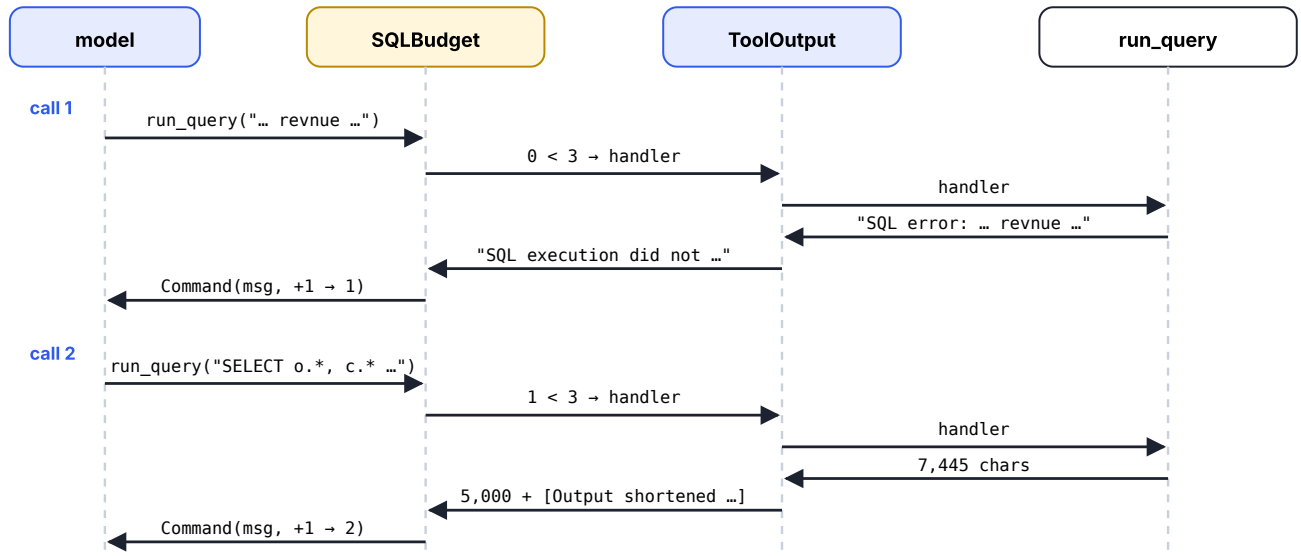


Figure 8. Two `run_query` calls through both wrappers. Values are illustrative.

In both calls the order is the same: budget in, cleaner in, tool, cleaner out, budget out. The tool's raw text never reaches the model directly. Both calls are counted, including the one that returned an error.

```
result["messages"] (run_query part only):
  ToolMessage("SQL execution did not return business data. SQL error: column \"revnue\" ...")
  ToolMessage("order_id | ... (5,000 characters) ...
               [Output shortened to 5000 characters. ... do not infer missing rows.]")
result["sql_call_count"]  → 2
```

10 What the cleaner does not do

Not its job	Why, or who does it
Change lookup, validation or web output	it returns every other tool's result unchanged; a long `web_search` result is not cut here
Decide whether a call may run	that is the budget's job, in the outer layer
Summarise or pick the important rows	it cuts at a character count; smarter selection (e.g. a `LIMIT`, aggregation) is the SQL's job
Count tokens	5,000 is characters; the token count depends on the model
Change anything in state except the message text	other `Command` keys pass through untouched
Fix the error	it labels the error clearly; the model still has to fix its SQL

> These lines match the “Does NOT” block at the top of the class in the notebook. Writing down what a component does not do is as useful as describing what it does: it stops people from relying on it for the wrong job.

11 Two middleware, two jobs

	`SQLBudgetMiddleware`	`ToolOutputMiddleware`
Question it answers	may this call run?	what should the model read?
Layer	outer	inner, next to the tool
Moves used	pass through · block · add to result	change the result
Touches state	yes: `sql_call_count`	no
Acts on	`run_query` only	`run_query` only
Setting	`max_sql_calls=3`	`max_output_chars=5000`

They could have been one class. Keeping them separate means each one does one thing, can be tested alone, and can be reused alone. You could drop the budget and keep the cleaner, or the reverse, by editing one list.

12 The limitation, and what Version 8 changes

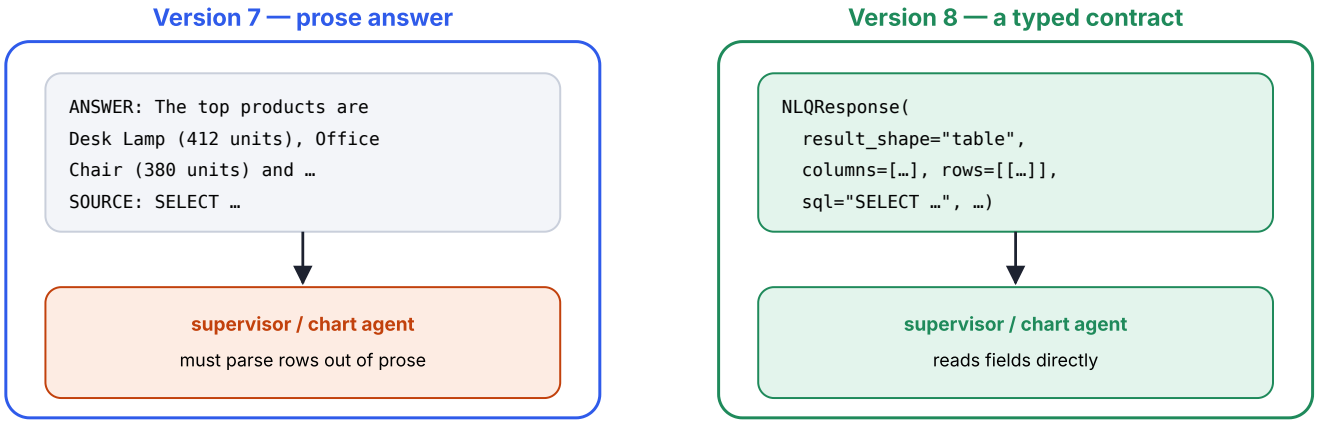


Figure 9. Version 7 answers people. Version 8 answers other programs.

Versions 2–7 end with a prose answer. That is right for a person reading it, but this agent is about to become one part of a bigger system: a supervisor routes questions to it, and a chart agent draws its results. Those programs need fixed fields, not paragraphs.

- **Rows in prose are retyped by the model.** A number can be copied wrong, and a program has to guess where the table starts.
- **The shape changes from answer to answer.** One answer has a table, the next a sentence.
- **Failure cases look alike.** “No rows”, “could not answer” and “never ran a query” all read as text.

	Version 7	Version 8
Final output	prose	`NLQResponse` (typed)
Rows come from	the model's text	`run_query`, saved in state
Model returns	the whole answer	only its judgment: `ModelDecision`
Middleware	budget + cleaner	unchanged — the cleaner already handles `Command`

13 Key takeaways

1. **A second tool wrapper sits inside the first.** Each wrapper's `handler` runs the next layer in; the innermost `handler` runs the tool.
2. **List order is nesting order.** First in the list is outermost: entry runs outside-in, exit runs inside-out.
3. **Gatekeepers outside, transformers inside.** A blocked call never reaches the cleaner; the cleaner sees the tool's raw output first.
4. **`ToolOutputMiddleware` only changes results.** It always calls `handler` first and never blocks.
5. **Four rules, first match wins:** empty → flag · error/refused → label · over 5,000 chars → cut + marker · otherwise unchanged.
6. **Say what happened before the raw text.** "SQL execution did not return business data" tells the model there are no numbers to report.
7. **Mark cut output.** "Do not infer missing rows" keeps a partial table from being treated as complete.
8. **Handle both `ToolMessage` and `Command`**, so Version 8 can reuse the class unchanged.
9. **Keep rules in plain methods** (`clean()`, `_process()`), so they can be tested without an agent.
10. **One job per middleware:** "may it run?" and "what does it return?" are separate classes.

> **Next — Version 8:** structured output. The model returns a small typed `ModelDecision`, `run_query` saves the real rows in state, and Python assembles an `NLQResponse` for the supervisor.